

50.007 Machine Learning

Lecture 2

Perceptron

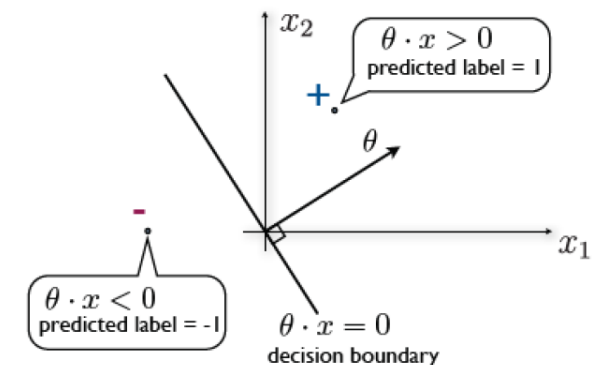
Linear Classifier

Linear Classification

- Linear classifier is a particular constrained set of classifiers

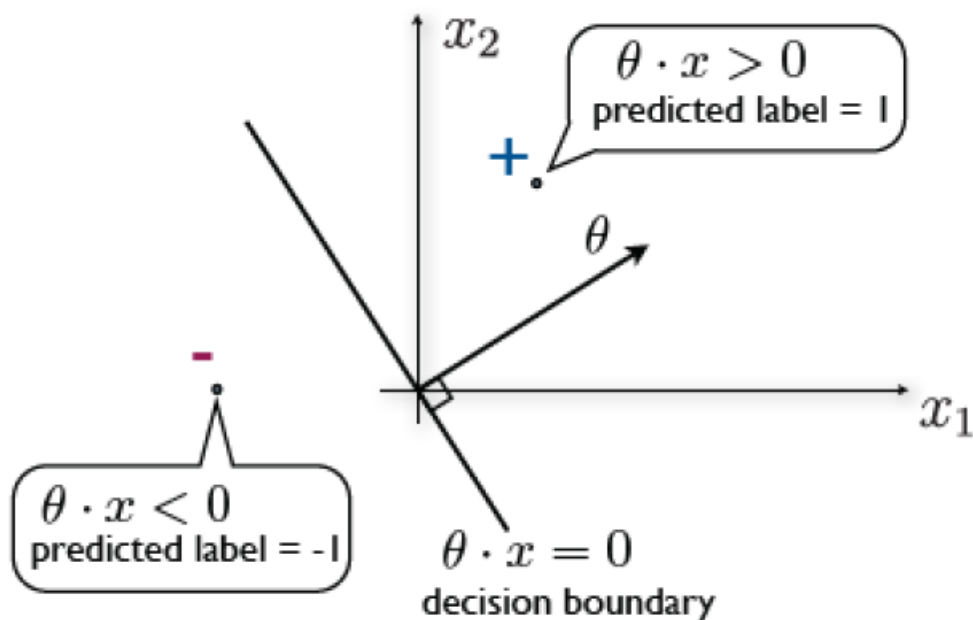
$$h(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d) = \text{sign}(\theta \cdot x) = \begin{cases} +1, & \theta \cdot x \geq 0 \\ -1, & \theta \cdot x < 0 \end{cases}$$

- where, $\theta \cdot x = \theta^T x$ and $\theta = [\theta_1, \dots, \theta_d]^T$ is a column vector of real valued parameters or weights
- Different settings of the **weights** give rise to **different classifiers**.

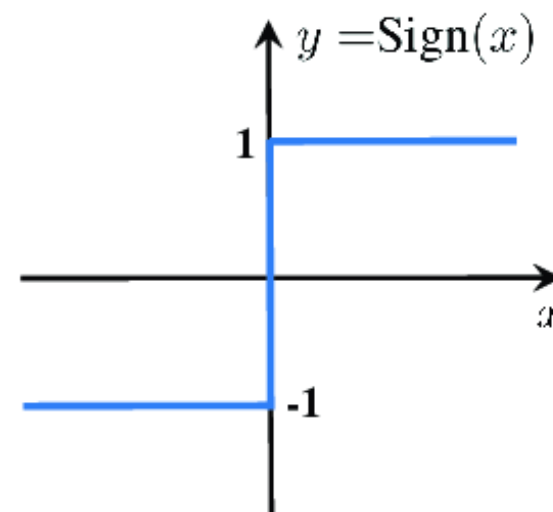


Linear Classification

$$h(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d) = \text{sign}(\theta \cdot x) = \begin{cases} +1, & \theta \cdot x \geq 0 \\ -1, & \theta \cdot x < 0 \end{cases}$$



* We do not update the decision boundary, instead we update the θ as since decision boundary is $\perp$ to θ to satisfy $\theta \cdot x = 0$, it will automatically change the decision boundary.



* Goal: To find an appropriate θ .

Linear Classifier

- Linear classifier through origin:

$$h(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d) = \text{sign}(\theta \cdot x) = \begin{cases} +1, & \theta \cdot x \geq 0 \\ -1, & \theta \cdot x < 0 \end{cases}$$

return a boolean 1 OR 0

- Training error:

$$\mathcal{E}_n(\theta) = \frac{1}{n} \sum_{t=1}^n \underbrace{\mathbb{I}[y^{(t)} \neq h(x^{(t)}; \theta)]}_{\text{blue bracket}} = \frac{1}{n} \sum_{t=1}^n \mathbb{I}[y^{(t)}(\theta \cdot x^{(t)}) \leq 0] \longrightarrow ?$$

- Linear classifier that achieves zero training error is called **realizable**.

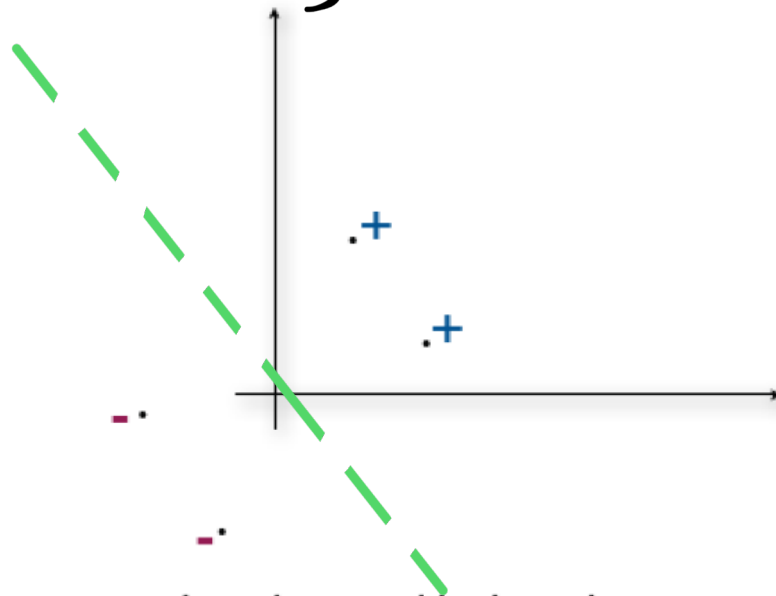
↓

If the output $y^{(t)}$ = supervised learning so we know the output
If it has not = $h(x^{(t)}; \theta)$ which is the prediction $\Rightarrow$ return 1
If it is = ; $\Rightarrow$ return 0 $\rightarrow$ no error $\hookrightarrow$ has error

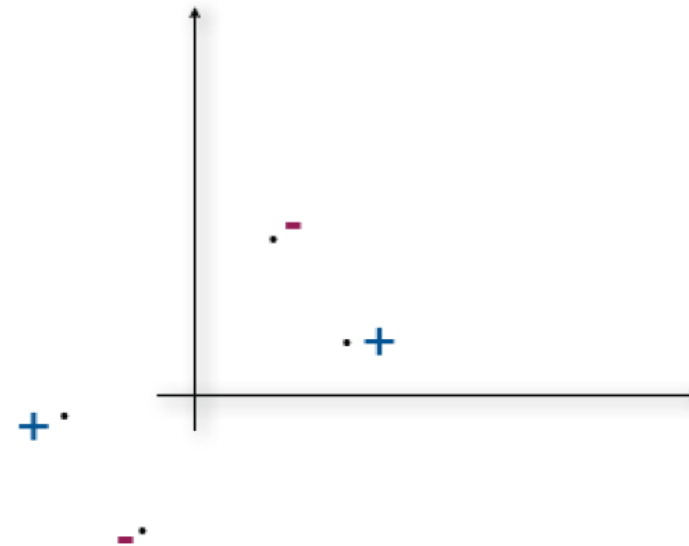
Linear Classifier

Definition 1.1 Training examples $S_n = \{(x^{(t)}, y^{(t)}), t = 1, \dots, n\}$ are linearly separable through origin if there exists a parameter vector $\hat{\theta}$ such that $y^{(t)}(\hat{\theta} \cdot x^{(t)}) > 0$ for all $t = 1, \dots, n$.

possible decision boundary



linearly separable through origin



not linearly separable

How do we find the weights such that they **minimize** the training error?

If linearly separable $\Rightarrow$ minimize weight = 0

If not linearly separable $\Rightarrow$ week 2

Perceptron update rule

- Initialize the **weight** ($\theta = 0$).
- For each training example 't' in S_n , classify the instance
 - if the prediction was correct, continue
 - else, $\theta^{(k+1)} = \theta^{(k)} + y^{(t)}x^{(t)}$
- Terminate if the **training error** is zero (realizable) or a pre-determined number of iterations are completed (non-realizable).

$k = \# \text{ of updates} / \# \text{ of mistakes the algorithm has done}$

Perceptron update rule

the true output $\Rightarrow$ if suppose to be +
if suppose to be -
!!
will make the new θ more + / -.

What does the update rule ($\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$) do?

- If the classifier predicted an instance that was negative but it should have been positive...
 - Currently: $\theta \cdot x < 0$
 - Want: $\theta \cdot x \geq 0$
- Adjust the weights θ so that this function values becomes more positive.
- If the classifier predicted positive but it should have been negative, shift the weights so that the value becomes more negative.

* It is only a mistake if $y^{(t)}$ & $x^{(t)}$ have opposite signs.

Perceptron update rule

- If a classification mistake is made on sample 't', i.e.,

$y^{(t)}(\theta \cdot x^{(t)}) \leq 0 \longrightarrow$ if it is $= 0$, it is on the decision boundary, $\therefore$ it is not recommended to have

- The updated weight vector is:

$$\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

- With updated weights, $y^{(t)}(\theta^{(k)} \cdot x^{(t)})$ becomes more positive and eventually becomes > 0 in a realizable case.

$$\begin{aligned} y^{(t)}(\theta^{(k)} + y^{(t)} x^{(t)}) \cdot x^{(t)} &= y^{(t)}(\theta^{(k)} \cdot x^{(t)} + y^{(t)} x^{(t)} \cdot x^{(t)}) \\ &= y^{(t)}(\theta^{(k)} \cdot x^{(t)}) + (y^{(t)})^2 (x^{(t)} \cdot x^{(t)}) \\ &= y^{(t)}(\theta^{(k)} \cdot x^{(t)}) + \|x^{(t)}\|^2 \end{aligned}$$

? disappear

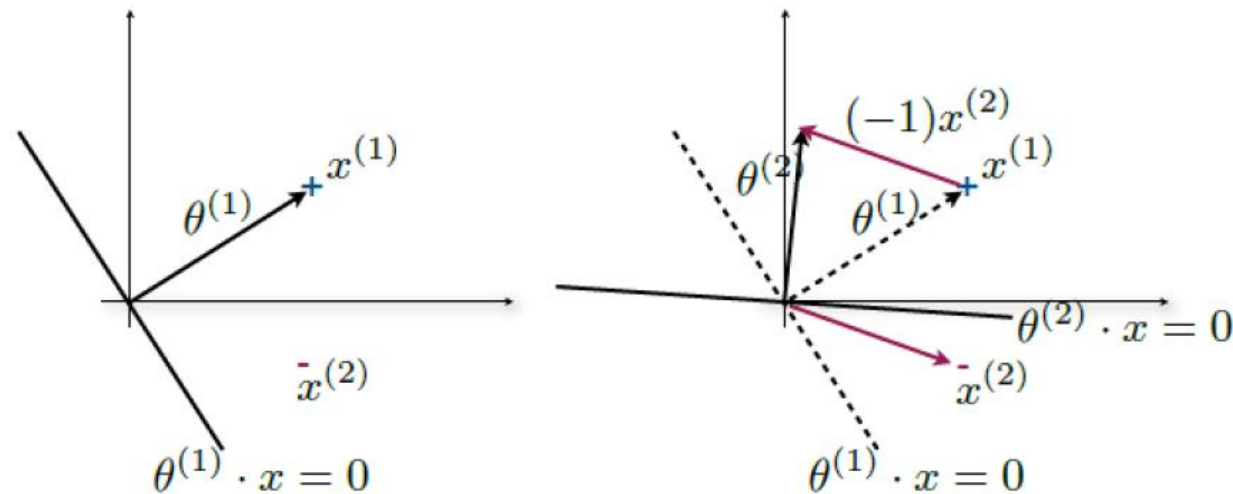
Perceptron update rule

- **Example:** Given the update rule, $\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$

$$\theta^{(1)} = \theta^{(0)} + x^{(1)}$$

$$\theta^{(2)} = \theta^{(1)} + (-1)x^{(2)}$$

?

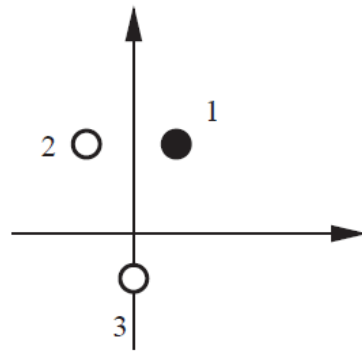


Exercise 1

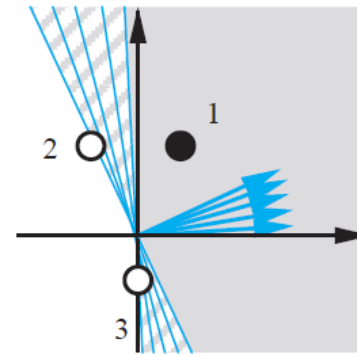
Perceptron update rule

Theorem 2.1 *The perceptron update rule converges after a finite number of mistakes when the training examples are linearly separable through origin.*

- **Test example:**



Input data

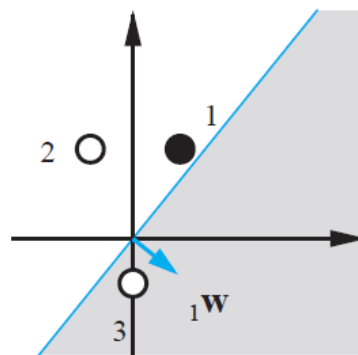
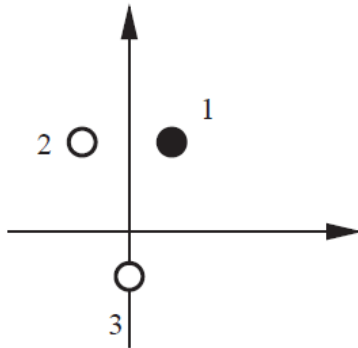


**Weight vectors
representing allowable
decision boundaries**

Perceptron update rule

$$y^{(1)}(\theta^0 \cdot x^{(1)}) = 1(1 \times 1) + (-0.8 \times 2) \\ = 1(1 - 1.6) \\ = 1(-0.6) \\ = -0.6 < 0$$

• Test example:



$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y^{(1)} = 1$$

$$x^{(2)} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, y^{(2)} = -1$$

$$x^{(3)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, y^{(3)} = -1$$

Perceptron update rule:

$$\text{if } y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 0 \quad \text{then} \\ \theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

$$\theta^{(0)} = \begin{bmatrix} 1 \\ -0.8 \end{bmatrix} \rightarrow \text{Random weight initialization}$$

→ Check if the point is classified correctly:

$$y^{(1)}(\theta^{(0)} \cdot x^{(1)}) = (+1)((1 * 1) + (-0.8 * 2)) = (1 - 1.6) = -0.6 < 0$$

→ Update $\theta^{(0)}$:

$$\theta^{(1)} = \theta^{(0)} + y^{(1)} x^{(1)} = \begin{bmatrix} 1 \\ -0.8 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1.2 \end{bmatrix}$$

$$\therefore \text{update } \theta^0 \text{ with } \theta^{(1)} = \theta^{(0)} + y^{(1)} x^{(1)} \quad \theta^{(1)} = \begin{bmatrix} 2 \\ 1.2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ -0.8 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 \\ 1.2 \end{bmatrix}$$

$$\text{check again: } y^{(1)} (\theta^{(1)} \cdot x^{(1)}) \leq 0$$

$$y^{(1)} (\theta^{(1)} \cdot x^{(1)}) = 1 ((2 \times 1) + (1.2 \times 2))$$

$$= 2 + 2.4 = 4.4 > 0 \quad \checkmark$$

$$y^{(2)} (\theta^{(1)} \cdot x^{(2)}) \leq 0$$

$$= -1 ((2 \times -1) + (1.2 \times 2)) = -1 (-2 + 2.4) = -1 (0.4) = -0.4 < 0 \quad \times$$

$$\theta^{(1)} = \begin{bmatrix} 2 \\ 1.2 \end{bmatrix} \quad x^{(2)} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

update $\theta^{(1)}$ with $\theta^{(2)} = \theta^{(1)} + y^{(2)} x^{(2)}$

$$= \begin{bmatrix} 2 \\ 1.2 \end{bmatrix} + (-1) \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 \\ 1.2 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$= \begin{bmatrix} 3 \\ -0.8 \end{bmatrix}$$

check with: $y^{(2)} (\theta^{(2)} \cdot x^{(1)}) \leq 0$

$$= -1 \left(\begin{bmatrix} 3 \\ -0.8 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ 2 \end{bmatrix} \right)$$

$$= -1 ((3 \times -1) + (-0.8 \times 2))$$

$$= -1 ((-3) + (-1.6)) = -1(-4.6) = 4.6 > 0 \checkmark$$

check for $y^{(1)} (\theta^{(2)} \cdot x^{(3)}) \leq 0$

$$= -1 \left(\begin{bmatrix} 3 \\ -0.8 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right)$$

$$= -1 \left((3 \times 0) + (-0.8 \times -1) \right)$$

$$= -1 (0 + 0.8)$$

$$= -0.8 < 0$$

update $\theta^{(2)}$ to $\theta^{(3)} = \theta^{(2)} + y^{(3)} x^{(3)}$

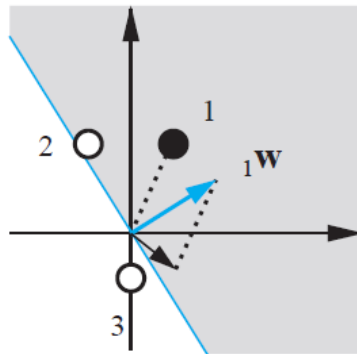
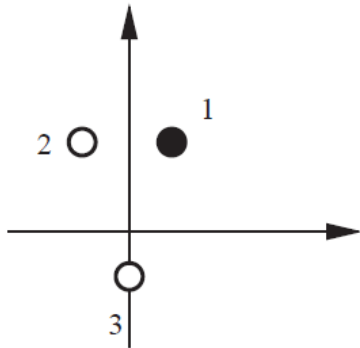
$$= \begin{bmatrix} 3 \\ -0.8 \end{bmatrix} + (-1) \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 \\ -0.8 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 \\ 0.2 \end{bmatrix}$$

Perceptron update rule

- **Test example:**



$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y^{(1)} = 1$$

$$x^{(2)} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, y^{(2)} = -1$$

$$x^{(3)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, y^{(3)} = -1$$

Perceptron update rule:

$$\text{if } y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 0 \quad \text{then} \\ \theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

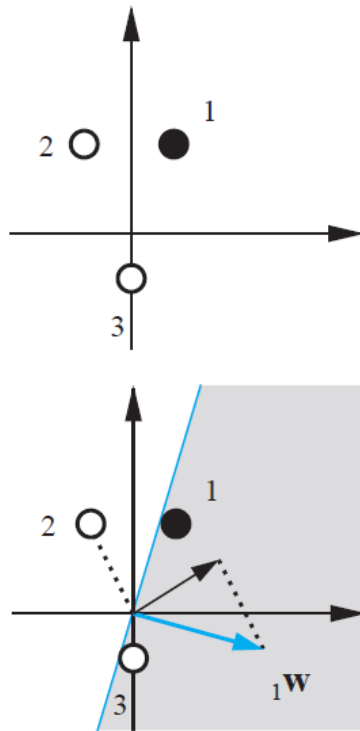
$$\theta^{(1)} = \begin{bmatrix} 2 \\ 1.2 \end{bmatrix}$$



1st Update

Perceptron update rule

- Test example:



$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y^{(1)} = 1$$

$$x^{(2)} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, y^{(2)} = -1$$

$$x^{(3)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, y^{(3)} = -1$$

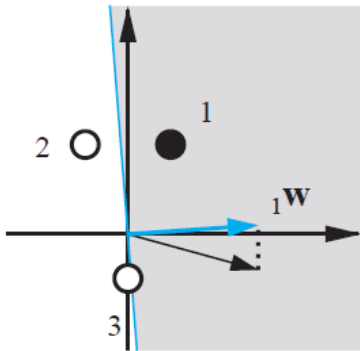
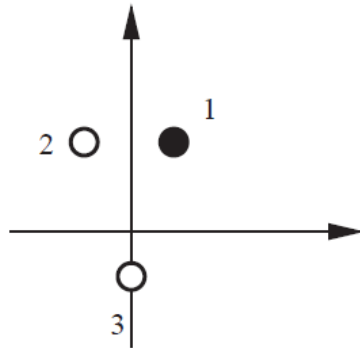
Perceptron update rule:

$$\text{if } y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 0 \quad \text{then} \\ \theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

$$\theta^{(2)} = \begin{bmatrix} 3 \\ -0.8 \end{bmatrix} \rightarrow \text{2nd Update}$$

Perceptron update rule

- Test example:



$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y^{(1)} = 1$$

$$x^{(2)} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, y^{(2)} = -1$$

$$x^{(3)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, y^{(3)} = -1$$

Perceptron update rule:

if $y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 0$ then
$$\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

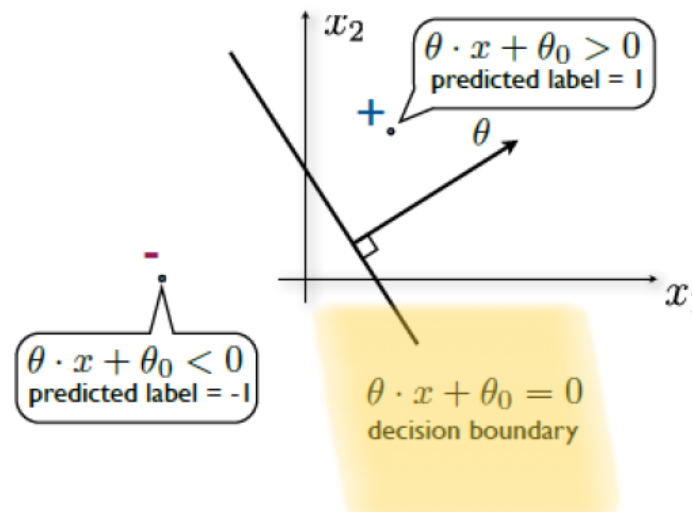
$$\theta^{(3)} = \begin{bmatrix} 3 \\ 0.2 \end{bmatrix} \rightarrow \text{3rd Update}$$

Linear Classifier (with offset)

Linear Classifier with offset

$$h(x; \theta, \theta_0) = \text{sign}(\theta \cdot x + \theta_0) = \begin{cases} +1, & \theta \cdot x + \theta_0 \geq 0 \\ -1, & \theta \cdot x + \theta_0 < 0 \end{cases}$$

- The hyper-plane $\theta \cdot x + \theta_0 = 0$ is **oriented parallel** to $\theta \cdot x = 0$
- Whereas, θ is still **orthogonal** to the decision boundary and $\theta_0 < 0$



* Since we move the decision boundary in the +ve direction, θ_0 is -ve since $\theta \cdot x + \theta_0 = 0$
 $\theta \uparrow = \theta \cdot x \uparrow \Rightarrow \therefore \theta_0 \downarrow$
 $\therefore \theta \cdot x + \theta_0 = 0$

Exercise 2

Linear Classifier with offset

- **Example:** Suppose we want to **predict** whether a web user will click on an ad for a refrigerator
- **Four features:**
 - Recently searched “refrigerator repair”
 - Recently searched “refrigerator reviews”
 - Recently bought a refrigerator
 - Has clicked on any ad in the recent past
- These are all **binary features** (values can be either 0 or 1)

Linear Classifier with offset

- Suppose these are the weights, θ

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- Evaluate the linear classifier with offset

$$h(x; \theta, \theta_0) = \begin{cases} +1, & \theta \cdot x + \theta_0 \geq 0 \\ -1, & \theta \cdot x + \theta_0 < 0 \end{cases}$$

Linear Classifier with offset

- **User 1:** Suppose these are the weights, θ (highlighted weight indicates that the feature value is 1)

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- $$\theta \cdot x + \theta_0 = (2 * 0) + (8 * 1) + (-15 * 0) + (5 * 0) + (-9)$$
$$= 8 - 9 = -1$$

Handwritten annotations:
A bracket under the first four terms is labeled $\theta \cdot x$.
An arrow points from the last term (-9) to the text $\text{offset} = \theta_0$.

- Prediction = No

Linear Classifier with offset

- **User 2:** Suppose these are the weights, θ (highlighted weight indicates that the feature value is 1)

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- $\theta \cdot x + \theta_0 = (2 * 1) + (8 * 1) + (-9) = 1$
- Prediction = Yes

Linear Classifier with offset

- **User 3:** Suppose these are the weights, θ (highlighted weight indicates that the feature value is 1)

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- $\theta \cdot x + \theta_0 = (8 * 1) + (5 * 1) + (-9) = 4$
- Prediction = Yes

Linear Classifier with offset

- **User 4:** Suppose these are the weights, θ (highlighted weight indicates that the feature value is 1)

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- $\theta \cdot x + \theta_0 = (8 * 1) + (-15 * 1) + (5 * 1) + (-9) = -11$
- Prediction = No
- If someone bought a refrigerator recently, they probably aren't interested in shopping for another one anytime soon.

Linear Classifier with offset

- **User 5:** Suppose these are the weights, θ (highlighted weight indicates that the feature value is 1)

Searched “repair”	2.0
Searched “reviews”	8.0
Recent purchase	-15.0
Clicked ads before	5.0
Offset	-9.0

- $\theta \cdot x + \theta_0 = 0 - 9$
- Prediction = No
- Since most people don’t click ads, the “default” prediction is that they will not click (the intercept pushes it to negative).

Exercise 3

Linear Classifier with offset

- If training examples are linearly separable through origin, they are also linearly separable with offset. Is the converse true? Why not?

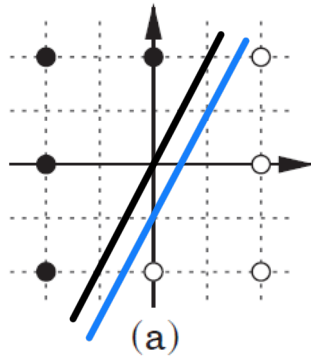
linearly separable through origin = linearly separable with offset

converse: linearly separable with offset = linearly separable through origin?

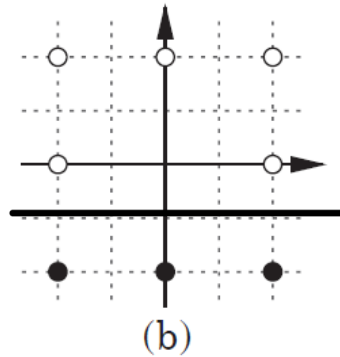
False. If the training examples are linearly separable with offset, there may exist points that cannot be linearly separable through the origin, see examples below:

Linear Classifier with offset

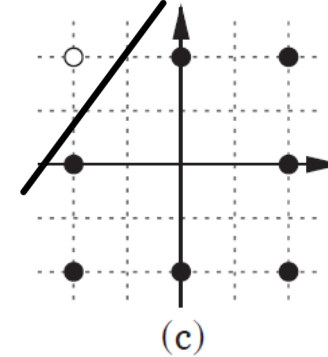
- If training examples are linearly separable through origin, they are also linearly separable with offset. Is the converse true? Why not?



yes. Both
with offset
& origin



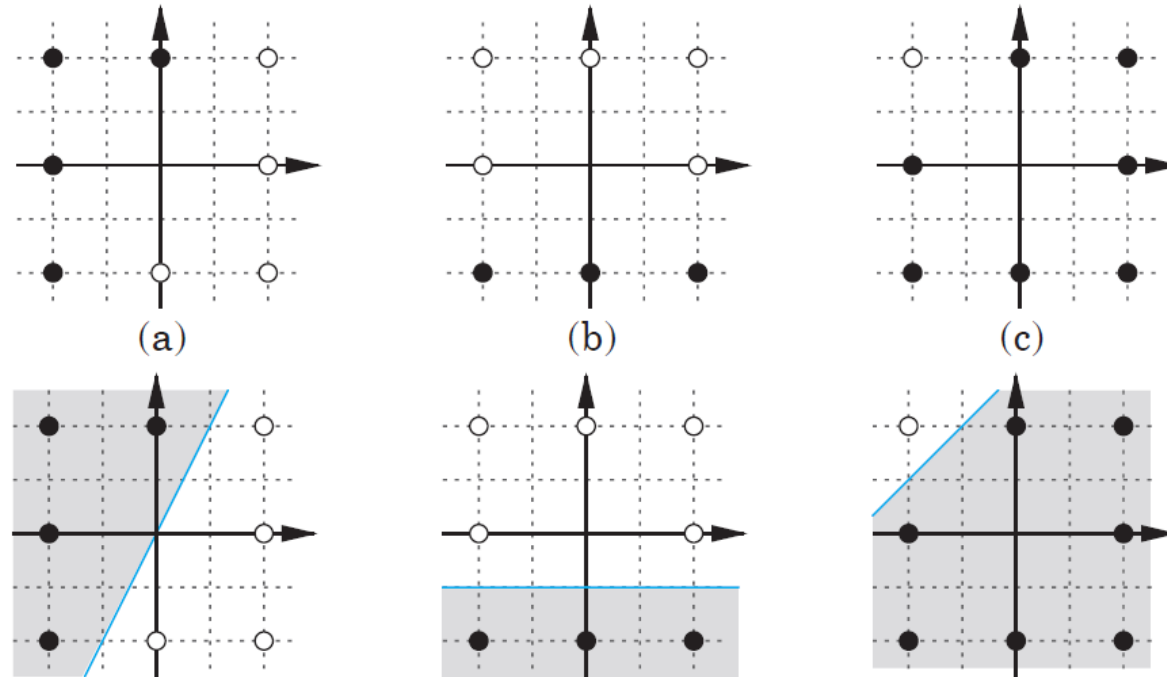
no. But
with offset
yes.



No. But
with offset
yes.

Linear Classifier with offset

- If training examples are linearly separable through origin, they are also linearly separable with offset. Is the converse true? Why not?



Linear Classifier with offset

Perceptron update rule:

- Initialize the **weight** ($\theta = 0$, $\theta_0 = 0$).
- For each training example 't' in S_n , classify the instance
 - if correct, continue
 - else, $\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$
 $\theta_0^{(k+1)} = \theta_0^{(k)} + y^{(t)}$
- Terminate if the **training error** is zero (realizable) or a pre-determined number of iterations are completed (non-realizable).

Intended Learning Outcomes

- Given a set of training examples, find out if they are **linearly separable (with and without offset)**.
- How does **Perceptron algorithm** work and apply to select the best classifier in a **realizable case**.
- What is the **guarantee of the Perceptron algorithm** when the dataset is linearly separable.
- Application of the **linear classifier to real data**.